

AdaDec: A Uncertainty-Guided Lookahead Decoding Framework for LLM-Based Code Generation

KAIFENG HE, Sun Yat-sen University, China

MINGWEI LIU, Sun Yat-sen University, China

CHONG WANG, Nanyang Technological University, Singapore

ZIKE LI, Sun Yat-sen University, China

YANLIN WANG, Sun Yat-sen University, China

XIN PENG, Fudan University, China

ZIBIN ZHENG, Sun Yat-sen University, China

Code generation with large language models (LLMs) is highly sensitive to token selection during decoding, particularly at uncertain decision points that influence program logic. While standard strategies such as greedy decoding treat all tokens uniformly, they overlook code-specific uncertainty patterns, leading to suboptimal performance. This paper presents an empirical study revealing that many generation errors stem from token ranking mistakes at high-uncertainty steps, where the correct token is present but not top-ranked.

Motivated by these findings, we propose ADADEC, a lookahead-based uncertainty-guided adaptive decoding framework that integrates a token-level *pause-then-rerank* mechanism driven by token uncertainty. ADADEC learns model-specific uncertainty thresholds and applies a lookahead-based reranking strategy when uncertainty is high. Experiments on HumanEval+, MBPP+, and DevEval benchmarks show that ADADEC improves Pass@1 accuracy by up to 20.9% in absolute terms over greedy decoding. More importantly, it consistently outperforms both competitive baselines like Beam Search and state-of-the-art adaptive decoding methods such as AdapT, while maintaining high efficiency through selective, uncertainty-triggered pausing. Our results highlight the promise of uncertainty-aware adaptive decoding for improving both the reliability and efficiency of LLM-based code generation.

CCS Concepts: • **Software and its engineering** → **Automatic programming**.

Additional Key Words and Phrases: Code Generation, Adaptive Strategy, Language Model

ACM Reference Format:

Kaifeng He, Mingwei Liu, Chong Wang, Zike Li, Yanlin Wang, Xin Peng, and Zibin Zheng. 2026. AdaDec: A Uncertainty-Guided Lookahead Decoding Framework for LLM-Based Code Generation. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE135 (July 2026), 23 pages. <https://doi.org/10.1145/3808142>

1 Introduction

Large language models (LLMs) have significantly advanced the field of code generation by learning expressive representations of programming syntax and semantics from large-scale corpora of

*Corresponding authors: Mingwei Liu and Chong Wang.

Kaifeng He, Mingwei Liu, Zike Li, Yanlin Wang, and Zibin Zheng are with the School of Software Engineering and the Zhuhai Key Laboratory of Trusted Large Language Models, Sun Yat-sen University, Zhuhai, China. Email: hekaifeng70@gmail.com, liumw26@mail.sysu.edu.cn, lizk8@mail2.sysu.edu.cn, yanlin-wang@outlook.com, zhizbin@mail.sysu.edu.cn.

Chong Wang is with Nanyang Technological University, Singapore. Email: chong.wang@ntu.edu.sg.

Xin Peng is with Fudan University, Shanghai, China. Email: pengxin@fudan.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/7-ARTFSE135

<https://doi.org/10.1145/3808142>

source code and documentation [2, 4, 14, 20, 22, 37]. Models such as CodeLlama [32] and DeepSeek-Coder [8]—pre-trained on billions of lines of code—exhibit strong capabilities in producing syntactically correct and context-aware code snippets from natural language prompts [21, 35, 38, 41]. These capabilities have catalyzed a range of intelligent software engineering tasks, including code completion, automated bug fixing [7, 15, 39], and test-case generation, ultimately reducing developer effort and accelerating the software development process [12, 13, 24].

At the core of code generation with large language models (LLMs) lies the decoding process, which translates learned latent representations into discrete, executable token sequences [3]. Decoding for code generation, however, faces domain-specific challenges. Code exhibits distinctive patterns of token uncertainty: certain tokens (e.g., identifiers or control keywords at the start of a line) are intrinsically ambiguous because their correct choice depends on **future logic**, while others (e.g., syntactically constrained keywords or repeated identifiers) are relatively deterministic. Crucially, mistakes at a few high-uncertainty points can yield irreversible semantic errors (for example, selecting “for” versus “return” at line start fundamentally alters control flow). Figure 1 provides an example. Therefore, the choice of decoding strategy—how the next token is selected or processed—plays a decisive role in balancing efficiency, diversity, and reliability.

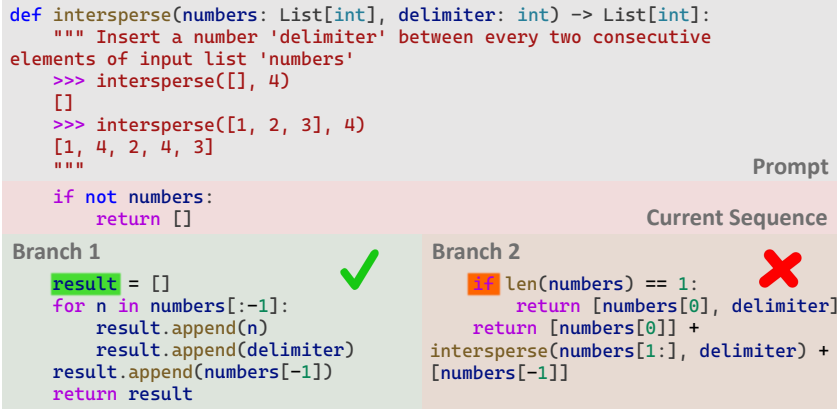


Fig. 1. Illustration of irreversible semantic errors in code due to high-uncertainty points.

A range of strategies have been proposed: deterministic greedy decoding [30] is simple and efficient but prone to premature commitment to suboptimal tokens; beam search [6] improves results by maintaining multiple candidates but incurs high computational overhead; sampling-based approaches such as top- k [10] and nucleus (top- p) [10] sampling enhance diversity but often introduce syntactic or semantic errors that are unacceptable for code; and uncertainty-based methods aim to exploit token-level difficulty signals. For instance, AdapT [45] estimates uncertainty by computing cross-entropy loss between model predictions and ground truth tokens, finding that tokens with higher loss values tend to cluster at line starts. Based on this *pre-identified* pattern rather than dynamic signals during decoding, AdapT assigns higher temperatures to line-initial tokens, but its reliance on randomness makes gains unstable and limited. By contrast, Uncert-CoT [44] dynamically measures uncertainty at decoding time, employing Shannon entropy [34] or probability difference (the gap between the top-1 and top-2 token probabilities) to decide whether a line-initial token is highly uncertain, in which case single-line chain-of-thought reasoning is triggered. While this avoids uniform treatment of all line starts, it overlooks uncertain tokens beyond line boundaries and has shown only marginal improvements in practice. Overall, existing methods either treat all decoding steps uniformly or rely on coarse heuristics tied to line structure,

and current uncertainty-based decoding approaches have not yet explored which uncertainty signals are most appropriate to guide intervention at decoding time.

To address this gap, we conduct an empirical study to better understand token-level uncertainty during the decoding process in LLM-based code generation. Using DeepSeek-Coder [8] and the HumanEval dataset [3], we analyze decoding failures by comparing model predictions that fail test cases with their corresponding ground-truth solutions. **Finding 1:** The analysis reveals that many errors stem from local ranking mistakes—where the correct token exists in the candidate set but is not ranked first—thereby causing logic drift. To further investigate this phenomenon, we perform next-token prediction on 164 HumanEval problems, recording decoding information and the rank of the ground-truth token at each step. This enables us to statistically identify the locations of logic drift points and analyze the relationship between logic drift and entropy. **Finding 2:** The results suggest that entropy can serve as a useful signal for detecting logic drift and guiding adaptive pausing and reranking. However, due to variability across models, a fixed entropy threshold is suboptimal, highlighting the need for a dynamic, model-aware thresholding mechanism.

Based on these findings, we introduce ADADEC, an uncertainty-guided adaptive decoding framework for code generation with large language models. ADADEC enhances standard decoding (e.g., greedy decoding, sampling) by incorporating a **pause-then-rerank** mechanism that reacts to model uncertainty, measured by Shannon entropy at each step. When uncertainty exceeds a threshold, decoding is paused and a reranking procedure is triggered. Because a fixed threshold does not generalize well across different models, ADADEC learns this threshold through a data-driven logistic regression approach. During reranking, ADADEC selects the top candidate tokens and evaluates them using a lookahead strategy inspired by A* search heuristics and a prior constrained generation method [27], scoring each candidate based on expected future continuations. The highest-scoring token is chosen for generation, and the process repeats until the sequence is complete. In contrast to prior approaches that either treat all decoding steps uniformly or rely on coarse heuristics tied to line structure, ADADEC systematically explores Shannon entropy as a token-level uncertainty signal and applies dynamic, token-level intervention across both line-initial and in-line tokens only at high-uncertainty steps, addressing the identified gap and enabling targeted correction of the most consequential decision points.

We conduct extensive experiments on the HumanEval+, MBPP+ and DevEval benchmarks using models ranging from 0.6B to 8B parameters. Our uncertainty-guided adaptive decoding framework, ADADEC, consistently improves Pass1 accuracy by up to 20.9% over Greedy decoding and consistently outperforms state-of-the-art adaptive decoding methods such as AdapT. In practice, ADADEC selectively pauses decoding at a low rate (averaging 6.75%), and the targeted lookahead introduces moderate additional latency, effectively balancing quality and efficiency. Ablation studies confirm that the learned entropy threshold accurately identifies uncertain decoding steps, and empirical comparisons show that using the learned threshold outperforms a fixed threshold, particularly for models where the fixed threshold deviates substantially from the learned value. Overall, ADADEC generalizes well across model scales and datasets, significantly improving code generation accuracy without incurring excessive runtime overhead.

In summary, this paper makes the following contributions:

- An empirical study that reveals many code generation errors arise from local token ranking mistakes, not only occurring at line beginnings but also sometimes within lines, and that Shannon entropy serves as a useful signal for detecting uncertain decoding steps prone to logic drift.

- An adaptive decoding framework, ADADEC, which employs an entropy-guided pause-then-rerank mechanism with learned, model-specific thresholds and a lookahead strategy to improve generation quality.
- Experimental results that show ADADEC improves Pass@1 accuracy by up to 20.9% over Greedy decoding, achieves high efficiency by selectively pausing decoding at a low rate, and generalizes effectively across diverse model sizes and benchmark datasets.

2 Empirical Study

We conduct an empirical study to investigate the characteristics of token uncertainty in LLM-based code generation and analyze the feasibility of integrating token uncertainty into decoding process. Specifically, the study aims to answer the following research questions:

- **RQ1:** Where do logic drifts occur in code, and how are they influenced by token uncertainty?
- **RQ2:** How can token uncertainty be leveraged to improve token selection during decoding?

2.1 Study Setup

2.1.1 Models. To ensure the generality of our findings, we conduct our study on a diverse set of open-source models ranging from 0.6B to 8B parameters, accessed via Hugging Face. These encompass various architectures and instruction-tuning schemes. Specifically, we include code-specialized models: DeepSeek-Coder-1.3B/6.7B-Instruct (DS-1.3B, DS-6.7B) [8], StableCode-Instruct-3B (ST-3B) [29], and Qwen2.5-Coder-1.5B/7B-Instruct (QW2.5-1.5B, QW2.5-7B) [11]. We also include general-purpose models featuring dynamic reasoning capabilities: the Qwen3 family, including the 0.6B (QW3-0.6B), 1.7B (QW3-1.7B), 4B (QW3-4B), and 8B (QW3-8B) variants [36].

2.1.2 Implementation. Our implementation is built with PyTorch and the Hugging Face Transformers library. Decoding is performed without batching (i.e., using a batch size of 1) to ensure independent processing of each instance. The maximum generation length is capped at 1,024 tokens for all methods to enable fair comparison across models.

Experiments are conducted on a machine equipped with NVIDIA RTX 5880 Ada Generation GPUs (48 GB VRAM each) and an NVIDIA A100 GPU (80 GB VRAM), supported by NVIDIA Driver 550.163.01 and CUDA 12.4.

2.2 RQ1: Where do logic drifts occur in code, and how are they influenced by token uncertainty?

To address this research question, we first use DS-6.7B as a representative model to generate code for 164 programming problems in the widely used HumanEval dataset [3]. We begin by analyzing the locations of *logic drifts*—points during decoding where the generated sequence diverges from the correct solution—which often lead to incorrect program behavior. We then investigate how token-level uncertainty, quantified using Shannon entropy [34], relates to the occurrence of such drifts.

2.2.1 Method. For each programming problem in the HumanEval dataset, we use DS-6.7B to generate code snippets using a greedy decoding strategy. We then filter out the generations that successfully pass all test cases. Based on an initial human analysis, we observe that in several cases, the model begins with a correct sequence but introduces a critical error—often in the form of incorrect code structures—at a specific step. For example, instead of generating a for loop to iterate over remaining elements, the model might incorrectly rank the return token at the top position, prematurely terminating the control flow. At these error points, the correct next token is not missing from the candidate set but is ranked below the top-1 position (commonly second

or third), resulting in an irreversible deviation from the correct logic. This suggests that some generation failures are caused by local ranking errors at specific decoding steps, ultimately leading to incorrect code behavior. We refer to such steps as logic *drift points* during decoding and aim to identify them.

Given the high cost of fully manual annotation, we develop a semi-automatic approach that combines abstract syntax tree (AST) comparison with human verification and fine-grained token-level annotation. To ensure annotation focuses only on faulty generations, we first filter out all generated samples that can pass the tests. Subsequently, we construct ASTs for both the ground-truth and generated code, and normalize them by removing identifiers and comments. This normalization ensures that the comparison focuses solely on structural differences. We then perform a parallel traversal to compare the two normalized ASTs node by node. For instance, a structural discrepancy is flagged when:

- (1) control-flow node types differ (e.g., `if` vs. `for`);
- (2) relational or boolean operators differ (e.g., `>` vs. `>=`);
- (3) loop or branching boundaries diverge;
- (4) indentation-aligned block structures differ.

Any identified mismatch is traced back to its corresponding line in the original code and extracted as a candidate drift point.

For each candidate line, we further determine whether the drift occurs at the beginning of the line and record this for statistical analysis. Finally, each candidate is reviewed by a human annotator to confirm the validity of the drift and to annotate the precise start token. To ensure consistency, annotators adhered to the following explicit guidelines:

- (i) focus strictly on structural (AST-level) differences;
- (ii) ignore lexical or superficial variations (e.g., variable names, comments);
- (iii) label the earliest divergence point;
- (iv) treat alternative valid implementations as non-drift unless the semantics fundamentally change.

For instance, a mismatch between the generated line `if shift > len(s):` and the ground-truth line `if shift >= len(digits):` triggers a drift point at the position of the operator `>=`.

In addition to the identified drift points, we collect all other non-drift decoding steps from the same generated code snippets. We compute the Shannon entropy for each step and compare the entropy distribution of drift points with that of non-drift steps. This comparison allows us to investigate the relationship between logic drift and token uncertainty, as measured by Shannon entropy. Given a probability distribution $\mathbf{p}$, the corresponding entropy $H(\mathbf{p})$ is calculated as follows:

$$H(\mathbf{p}) = - \sum_{p_i \in \mathbf{p}} p_i \log p_i$$

2.2.2 Results. We identify a total of 46 drift points and 2,889 non-drift decoding steps across 46 HumanEval problems. Statistical analysis shows that divergences at line start account for 86.96% of all drift points, while 13.04% occur within lines. This observation suggests that focusing solely on line-initial tokens for uncertainty-based intervention is insufficient, as it ignores more than 10% of logic drift points. Figure 2 presents a boxplot comparing the entropy distribution at drift points with that of all non-drift steps. For clarity, outliers are omitted in boxplots. The entropy values at drift points are markedly higher: the median entropy is 1.26, with a mean of 1.39, compared to a much lower median of 0.03 and mean of 0.30 for the rest. The interquartile range (IQR) for drift points is also shifted higher (0.63–2.01) relative to that of non-drift steps (0.003–0.34). The results

show that tokens at drift points exhibit significantly higher entropy values compared to the rest, suggesting a clear relationship between entropy and logic drift.

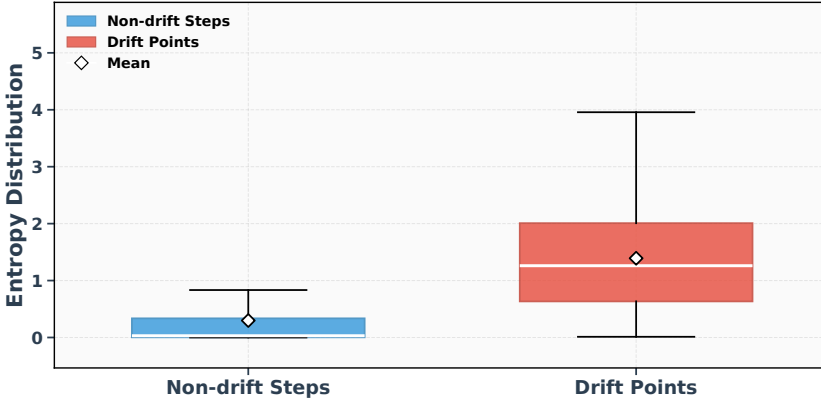


Fig. 2. Entropy comparison between drift points and non-drift decoding steps.

Finding 1: This empirical pattern indicates that focusing solely on line-start tokens for uncertainty-based intervention is insufficient, motivating the search for a suitable uncertainty signal that monitors all tokens and triggers targeted intervention at high-uncertainty steps. Subsequent analysis shows that token uncertainty measured by Shannon entropy is strongly correlated with logic drift, providing a useful signal for identifying potential drift points and predicting code behavior deviations during decoding.

2.3 RQ2: How can token uncertainty be leveraged to improve token selection during decoding?

To integrate token uncertainty into the token selection process and enhance the likelihood of generating correct code, we further investigate the relationship between entropy and the rank positions of expected tokens during decoding.

2.3.1 Method. We perform next-token prediction using 164 problems and their corresponding ground-truth solutions from the HumanEval dataset. Specifically, given a problem P and its ground-truth solution S , we tokenize them into sequences T^P and T^S , respectively. At each decoding step k , we feed the concatenated sequence $T^P \oplus T^S_{:k}$ into the LLM to obtain a probability distribution over the next token. Here, $T^S_{:k}$ denotes the prefix of the ground-truth solution containing the first k tokens, and $\oplus$ indicates sequence concatenation. By iterating over all positions k from 1 to the length of T^S , we collect a set of predicted probability distributions. For each prediction, we compute the Shannon entropy of the distribution and record the rank of the expected (ground-truth) token within it.

We apply this next-token prediction procedure to a range of open-source LLMs, including five code-specific models, DS-1.3B, DS-6.7B, ST-3B, QW2.5-1.5B, and QW2.5-7B, as well as four general-purpose models: QW3-0.6B, QW3-1.7B, QW3-4B, and QW3-8B. For Qwen3, we disable the reasoning-specific settings (e.g., thinking mode). In total, we collect 180,879 token-level probability distributions for two DeepSeek-Coder models, 169,496 for StableCode, and 129,165 for Qwen series.

2.3.2 Results. We compute the Spearman correlation between entropy and the rank of the ground-truth token across all models. As shown in Table 1, the results consistently exhibit a positive

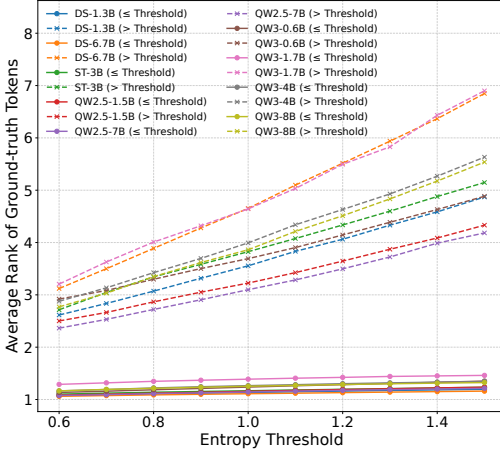


Fig. 3. Change in the average rank of ground-truth tokens above and below a given entropy threshold, as the threshold increases.

correlation, further reinforcing the notion that entropy serves as a meaningful signal for identifying uncertain decoding steps. We omit p-values from the table, as the large sample sizes (approximately 130,000 to 180,000 tokens per model) yield statistically significant results with p-values effectively indistinguishable from zero.

Table 1. Spearman correlation ρ between entropy and the rank of ground-truth tokens.

Model	DS-1.3B	DS-6.7B	ST-3B	QW2.5-1.5B	QW2.5-7B	QW3-0.6B	QW3-1.7B	QW3-4B	QW3-8B
ρ	0.4728	0.4595	0.4620	0.4906	0.4755	0.5249	0.5028	0.4936	0.4877

Figure 4 shows how the percentage of decoding steps with entropy exceeding a given threshold changes as the threshold increases. Figure 3 illustrates the corresponding change in the average rank of ground-truth tokens, separated by whether the entropy at each step is above or below the threshold. These plots reveal that different models exhibit varying sensitivity to entropy thresholds. For example, at a threshold of 1.2, QW3-0.6B has approximately 13% of decoding steps exceeding the threshold, while other models remain below 10%. In terms of average rank, two models (DS-6.7B and QW3-1.7B) exceed a value of 5.5, and the rest fall in the range of 3.5 to 4.7.

Finding 2: The observed correlation between entropy and the rank of the ground-truth token suggests that entropy can be used as an indicator to adaptively **pause** the decoding process and **rerank** uncertain tokens. However, our entropy percentile analysis shows that it is difficult to define a universal, fixed entropy threshold across all models that effectively balances pause frequency and the number of reranking candidates. This highlights the need for a **dynamic** mechanism to determine the model-specific entropy threshold.

3 Approach

We now introduce **ADADEC**, an uncertainty-guided adaptive decoding framework for LLM-based code generation. ADADEC integrates a token-level **pause-then-rerank** mechanism into the decoding process of the target LLM, drawing on insights from our empirical analysis.

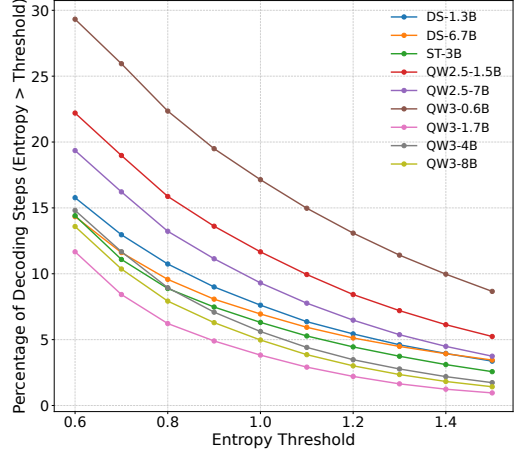


Fig. 4. Change in the percentage of decoding steps exceeding a given entropy threshold as the threshold increases.

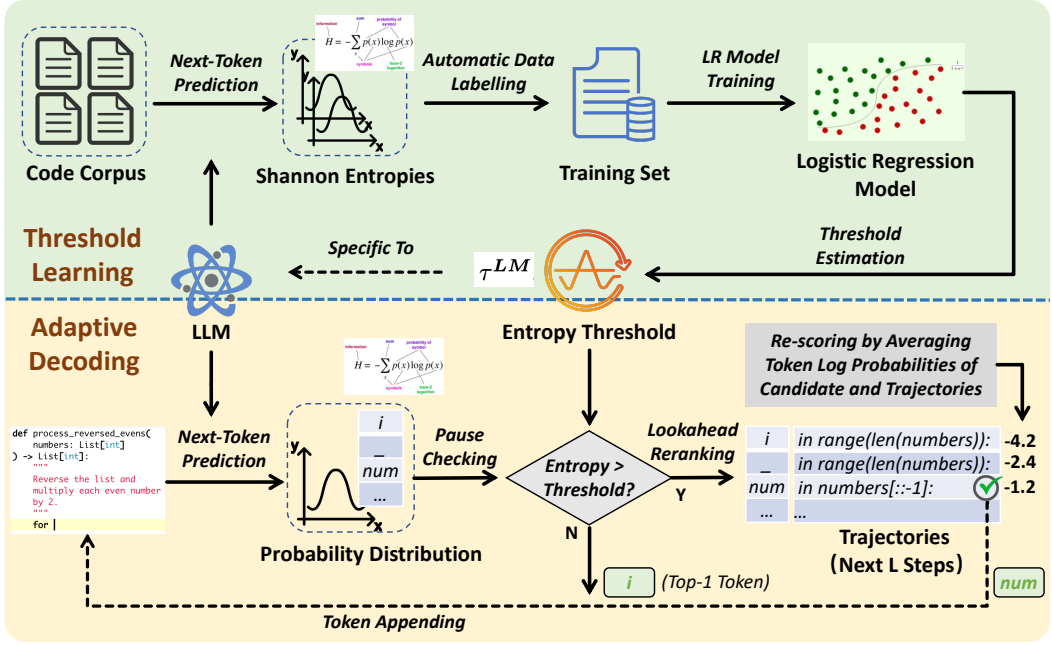


Fig. 5. Approach Overview of ADADEC

3.1 Overview

As illustrated in Figure 5, ADADEC is an adaptive (dynamic) decoding framework: at inference time it continuously monitors per-step uncertainty and dynamically decides whether to intervene, rather than applying a uniform policy across all steps or only at line-start positions. Given a target model $\mathbf{LM}$, at each decoding step t the model produces a next-token distribution $\mathbf{p}_t$. ADADEC computes the Shannon entropy $H(\mathbf{p}_t)$ and compares it against a model-specific threshold $\tau^{\mathbf{LM}}$. If $H(\mathbf{p}_t) \leq \tau^{\mathbf{LM}}$, the framework proceeds with the base decoding policy (e.g., greedy, top- k , or top- p sampling). If $H(\mathbf{p}_t) > \tau^{\mathbf{LM}}$, ADADEC pauses normal decoding and triggers a targeted reranking routine over the top- K candidates. During reranking, each candidate undergoes a short lookahead of up to N steps to estimate the quality of its future trajectory, and the candidate with the highest score is selected as the next token. This per-step decision loop repeats until an end-of-sequence (EOS) token is generated.

The adaptive nature of ADADEC has three practical consequences. First, the intervention is selective and data-driven: only high-uncertainty steps incur additional computation, which limits overhead compared to always-on strategies. Second, the threshold $\tau^{\mathbf{LM}}$ is learned in a preparation stage (we describe the learning procedure in Sec. 3.2) so that the pause decision is model-aware rather than hand-tuned. Third, the framework supports flexible reranking configurations (e.g., adjustable lookahead length N and lookahead width K), allowing trade-offs between accuracy and latency to be tuned for different deployment constraints.

ADADEC is designed to be broadly applicable. Since the pause-then-rerank mechanism is triggered purely by uncertainty signals (entropy), it can be seamlessly integrated with various decoding strategies, including sampling. This flexibility underscores the generality of ADADEC: rather than being tied to a specific decoding style, it acts as a plug-in framework that can enhance both deterministic (e.g., greedy) and stochastic (e.g., sampling) generation. Unless otherwise specified, we use **greedy decoding as the base policy** throughout the remainder of this paper.

The details of the pause checking and lookahead reranking are described in the following sections.

3.2 Entropy-based Pause Checking

We use the Shannon entropy $H(\mathbf{p}_t)$ over the probability distribution $\mathbf{p}_t$ predicted by the model **LM** to measure the token uncertainty. When H exceeds a threshold, ADADEC pauses the decoding process. As revealed in the empirical study, a fixed entropy threshold may not generalize well across models with different output distributions. Therefore, we propose a data-driven method to learn a model-specific threshold τ^{LM} using logistic regression.

3.2.1 Data Collection. We cast threshold learning as a binary classification task. Each decoding step is a sample, where the input feature is the entropy $H(\mathbf{p}_t)$ and the label is 1 if the ground-truth token is the top-1 prediction, and 0 otherwise. Training data is generated via ground-truth-guided next-token prediction on BigCodeBench [46]. To strictly mitigate data contamination, we verified semantic similarity between BigCodeBench and our evaluation sets using SBERT [31]. Manual review of all problem pairs exceeding a 0.7 cosine similarity threshold (maximum observed was 0.75) confirmed zero overlapping duplicates. Through this process, we collect approximately 100K–160K positive samples and 16K–23K negative samples per model (e.g., 161K positive and 19K negative for DS-6.7B).

We then balance the dataset by randomly downsampling the positive samples and selecting an equal number of negative samples, which is a common technique for handling imbalanced classification tasks [18]. This choice is driven by the nature of logic drift in code generation: once a drift point occurs, it typically leads to irreversible semantic errors. Therefore, it is essential to maximize the classifier’s ability to detect negative cases, even if they are relatively scarce. By constructing a balanced dataset, we prevent the model from being biased toward the majority class and enable it to more effectively learn the decision boundary for identifying uncertain decoding steps. This strategy enhances the classifier’s sensitivity to potential drift points, thereby improving the overall reliability of the entropy-based triggering mechanism.

3.2.2 Regression Model Training. We train a logistic regression model to predict whether the ground truth token is the top-1 prediction, using only the entropy value as input. The model estimates the probability of a positive label using the sigmoid function:

$$P(y = 1 \mid H^{(i)}) = \frac{1}{1 + \exp(-(\beta_0 + \beta_1 H^{(i)}))},$$

where $H^{(i)}$ denotes the entropy of the i -th sample, and β_0, β_1 are the model parameters to be learned. These parameters are learned by minimizing the binary cross-entropy loss through maximum likelihood estimation. Note that, since the training data is collected using the target **LM**, the trained logistic regression model is specific to that **LM**.

3.2.3 Threshold Estimation. To estimate a model-specific entropy threshold τ^{LM} , we use the trained logistic regression model to perform binary classification on the validation set, where the task is to predict whether the top-1 token matches the ground-truth token. As the logistic regression model outputs probabilities, we first identify the optimal probability threshold that maximizes classification accuracy, and then convert this probability threshold into an equivalent entropy threshold.

Specifically, we conduct a grid search over probability thresholds in the range [0.01, 0.99], using a step size of 0.01. For each candidate threshold, a sample is classified as positive if its predicted probability exceeds the threshold, and negative otherwise. The threshold that achieves the highest classification accuracy on the validation set is selected as the optimal probability threshold, denoted

by p^* . We then convert this optimal probability threshold into the corresponding entropy threshold using the parameters of the trained logistic regression model. Given the intercept β_0 and the coefficient β_1 , the entropy threshold τ^{LM} is computed using the log-odds transformation:

$$\tau^{LM} = \frac{\log\left(\frac{p^*}{1-p^*}\right) - \beta_0}{\beta_1}.$$

Entropy values below τ^{LM} are classified as positive (i.e., the model is likely to correctly predict the top-1 token), while those above are classified as negative. This entropy threshold is then used as the final decision boundary for the target language model during decoding.

3.2.4 Pause Triggering. During decoding, ADADEC uses the learned threshold τ^{LM} to decide when to trigger candidate reranking. If the entropy at a decoding step exceeds τ^{LM} , it indicates high uncertainty, suggesting that the top-1 prediction may be incorrect. In such cases, the decoding process is paused, and the top candidates are reranked to improve accuracy.

3.3 Lookahead-based Token Reranking

ADADEC employs a lookahead re-scoring strategy for token reranking, enhancing selection by assessing the future impact of each candidate token at every decoding step. Inspired by A* search heuristics and prior work on constrained generation [27], this method extends token probability evaluation beyond immediate next-token predictions to consider longer-term sequence quality. The process is governed by two hyperparameters: the *lookahead beam size* (B), which determines the number of candidate tokens considered, and the *lookahead length* (L), which sets the length of the future token sequence evaluated.

3.3.1 Lookahead Process. At decoding step t , the model outputs a probability distribution $\mathbf{p}_t$ over the vocabulary for the next token. From this distribution, we select the top- B tokens to form the candidate set $C_t = \{c_t^{(1)}, c_t^{(2)}, \dots, c_t^{(B)}\}$.

For each candidate token $c_t^{(k)}$, we construct a lookahead trajectory of length L by appending L tokens generated via greedy decoding, conditioned on the current input and previously generated tokens. This results in the trajectory $\mathbf{T}_t^{(k)}$:

$$\mathbf{T}_t^{(k)} = (c_t^{(k)}, \hat{c}_{t+1}^{(k)}, \hat{c}_{t+2}^{(k)}, \dots, \hat{c}_{t+L}^{(k)}),$$

where each $\hat{c}_{t+i}^{(k)}$ is the most probable token predicted at step $t+i$, given the prefix ending with $c_t^{(k)}$.

Trajectory Scoring. We evaluate each trajectory by computing the average log-probability of its tokens:

$$\text{score}(c_t^{(k)}) = \frac{1}{L+1} \left[\log P(c_t^{(k)}) + \sum_{i=1}^L \log P(\hat{c}_{t+i}^{(k)}) \right],$$

where $P(\cdot)$ denotes the model's predicted probability for a token. Because we apply an early stopping mechanism, different candidates may result in varying lookahead lengths. To ensure a fair comparison across such candidates, we normalize the total log-probability by computing the geometric mean of token probabilities in the above equation, rather than relying solely on log-likelihood. This normalization mitigates the bias against longer sequences, which tend to accumulate lower joint probabilities. The resulting score captures the model's overall confidence in the full trajectory, balancing immediate token likelihood with the coherence of future predictions.

Token Selection. The candidate with the highest trajectory score is selected as the output token at step t :

$$c_t^* = \arg \max_{c_t^{(k)} \in C_t} \text{score}(c_t^{(k)}).$$

This process is repeated at each decoding step, ensuring that token selection is guided not just by immediate probabilities, but by the expected quality of the downstream sequence.

This lookahead mechanism improves generation quality by favoring tokens that lead to fluent and coherent continuations, which is particularly important in structured domains like code generation. The hyperparameters B and L control the trade-off between computational cost and generation quality: larger values offer better foresight at the expense of increased inference time.

3.3.2 Lookahead Beam Size Selection. To determine an appropriate value for B , we analyze the training data collected in Section 3.2.1. Specifically, we examine the average rank of the ground truth token at decoding steps where the Shannon entropy is either above or below the threshold τ^{LM} , which is learned via logistic regression. After filtering out samples where the ground truth token's rank exceeds 20, we observe a clear distinction: when the entropy is below or equal to the threshold, the average rank typically falls between 1.1 and 1.3, indicating high model confidence. In contrast, when the entropy exceeds the threshold—thus triggering a pause—the average rank increases to between 2.6 and 3.4.

These findings support the use of a small beam size (e.g., $B = 3$) to achieve a practical trade-off between candidate coverage and computational efficiency.

3.3.3 Lookahead Length Selection. We adopt a lookahead length parameter L when performing trajectory scoring. At each reranking step we simulate up to L future tokens for each candidate to estimate the quality of its continuation. The parameter L controls the trade-off between accuracy and efficiency: a larger L provides a more reliable estimate of long-term trajectory quality but incurs higher computation cost, whereas a smaller L reduces overhead but may fail to capture meaningful differences among candidates.

In practice, we find that moderate values of L (e.g., $L = 5$) are sufficient to balance these concerns, since code generation typically exhibits strong local dependencies and short-horizon lookahead already provides useful signals. We analyze the effect of varying L through experiments in Section 4.5.

4 Evaluation

We conduct extensive experiments to evaluate the effectiveness of ADADec. Specifically, we aim to answer the following research questions:

- **RQ1 (Effectiveness):** How effective is ADADec in improving the functional correctness of generated code?
- **RQ2 (Efficiency):** How efficient is ADADec in terms of computational cost?
- **RQ3 (Ablation Study):** How effective is the logistic regression-based threshold learning component, and can the learned thresholds better guide practical decoding?
- **RQ4 (Parameter Sensitivity):** How does ADADec perform under different values of the lookahead length L ?

4.1 Experimental Setup

Our experimental setup is as follows.

Datasets. We evaluate our method on three widely used code generation benchmarks:

- **HumanEval+** [23]: An extended version of HumanEval[3] with test cases increased by a factor of 81 for 164 Python problems, enabling rigorous functional correctness evaluation.
- **MBPP+** [23]: A refined subset of 378 MBPP[1] problems with 35 times larger unit tests for stricter evaluation.
- **DevEval** [19]: A repository-level benchmark aligned with real-world scenarios. We randomly sample 238 problems from its 1,874 samples, ensuring 90% confidence within a $\pm 5\%$ margin.

Baselines. We compare ADADEC against three decoding strategies:

- **Greedy Decoding** [30]: Selects the highest-probability token at each step; computationally efficient but myopic.
- **Beam Search** [6]: Maintains the top- k sequences. We set beam size $B = 3$ to match ADADEC's candidate pool, isolating the benefits of uncertainty-guided reranking.
- **AdapT** [45]: Adjusts sampling temperature for line-start (high-uncertainty) tokens. We empirically set $a = 0.5$ and $b = 0.05$ (for other tokens) for optimal benchmark performance.

We also considered **Uncert-CoT** [44], another uncertainty-guided decoding method originally proposed for chain-of-thought reasoning tasks. However, we do not include it in our experimental comparison since no official implementation is available and the original paper omits several crucial details necessary for faithful reproduction.

4.2 RQ1: Effectiveness in Improving Functional Correctness

Table 2 presents the Pass@1 performance of four decoding strategies—Greedy, Beam Search ($B = 3$), AdapT, and ADADEC—across nine LLMs evaluated on the HumanEval+, MBPP+, and DevEval datasets.

Table 2. Pass@1 (%) comparison of Greedy, Beam Search, AdapT, and ADADEC on HumanEval+, MBPP+, and DevEval. Values in parentheses indicate the change relative to Greedy.

Model	HumanEval+				MBPP+				DevEval			
	Greedy	Beam	AdapT	ADADEC	Greedy	Beam	AdapT	ADADEC	Greedy	Beam	AdapT	ADADEC
DS-1.3B	62.20 (+0.00)	62.20 (+1.21)	63.41 (+3.04)	65.24 (+3.04)	52.12	51.59 (-0.53)	52.65 (+0.53)	53.17 (+1.05)	14.29	18.91 (+4.62)	12.61 (-1.68)	18.49 (+4.20)
DS-6.7B	71.34	76.83 (+5.49)	72.56 (+1.22)	73.78 (+2.44)	65.61	65.87 (+0.26)	66.93 (+1.32)	67.72 (+2.11)	25.21	28.57 (+3.36)	26.47 (+1.26)	26.47 (+1.26)
ST-3B	48.78	50.61 (+1.83)	51.83 (+3.05)	50.61 (+1.83)	38.89	45.77 (+6.88)	39.42 (+0.53)	46.03 (+7.14)	15.55	13.03 (-2.52)	13.03 (-2.52)	22.69 (+7.14)
QW2.5-1.5B	64.02	68.90 (+4.88)	64.02 (+0.00)	67.07 (+3.05)	59.79	61.64 (+1.85)	60.32 (+0.53)	62.17 (+2.38)	10.08	11.34 (+1.26)	10.50 (+0.42)	14.71 (+4.63)
QW2.5-7B	84.15	83.54 (-0.61)	87.20 (+3.05)	87.20 (+3.05)	71.16	71.96 (+0.80)	72.22 (+1.06)	72.22 (+1.06)	18.49	14.71 (-3.78)	18.07 (-0.42)	23.11 (+4.62)
QW3-0.6B	17.07	24.39 (+7.32)	18.29 (+1.22)	22.56 (+5.49)	9.52	21.16 (+11.64)	11.11 (+1.59)	30.42 (+20.90)	2.94	5.88 (+2.94)	2.52 (-0.42)	7.56 (+4.62)
QW3-1.7B	40.24	41.46 (+1.22)	42.07 (+1.83)	44.51 (+2.7)	37.57	43.39 (+5.82)	36.51 (-1.06)	42.86 (+5.29)	7.98	9.24 (+1.26)	7.56 (-0.42)	11.76 (+3.78)
QW3-4B	53.05	57.32 (+4.27)	53.66 (+0.61)	56.10 (+3.05)	40.48	42.86 (+2.38)	41.01 (+0.53)	51.32 (+10.84)	9.24	13.45 (+4.21)	9.24 (+0.00)	16.81 (+7.57)
QW3-8B	56.71	59.76 (+3.05)	57.32 (+0.61)	62.80 (+6.09)	29.37	36.24 (+6.87)	29.63 (+0.26)	47.09 (+17.72)	4.62	6.30 (+1.68)	4.20 (-0.42)	7.56 (+2.94)
AVG. Δ	–	+3.05	+1.42	+3.59	–	+4.00	+0.59	+7.61	–	+1.45	-0.47	+4.53

Comparison with Greedy Search. Across all three benchmarks, ADADEC consistently outperforms Greedy decoding. On HumanEval+, ADADEC improves Pass@1 by an average of **+3.59%**,

with gains as high as +6.09% for QW3-8B and +5.49% for QW3-0.6B. On MBPP+, the improvement is even more pronounced: ADADEC yields an average gain of **+7.61%**, with particularly significant performance boosts observed for QW3-0.6B (+20.90%) and QW3-8B (+17.72%). On DevEval, ADADEC demonstrates robust effectiveness, yielding meaningful improvements for all evaluated models (**+4.53%** on average). Particularly, QW3-4B and ST-3B show strong gains of +7.57% and +7.14%, respectively. Unlike the mixed results sometimes observed with other strategies, ADADEC achieves positive improvements across all models on DevEval, highlighting its ability to handle repository-level contexts where long-range logic consistency is crucial.

Comparison with Beam Search. A critical question is whether the gains of ADADEC stem merely from searching multiple candidate tokens or from our uncertainty-guided strategy. Comparing ADADEC with standard Beam Search (using the same beam width $B = 3$), we observe that ADADEC consistently achieves superior performance. While Beam Search provides a baseline improvement over Greedy (averaging +3.05% on HumanEval+ and +4.00% on MBPP+), ADADEC surpasses it by further margins. The advantage of ADADEC is particularly evident on the more challenging MBPP+ and DevEval benchmarks. For instance, on DevEval, Beam Search yields a modest average gain of +1.45%, whereas ADADEC achieves **+4.53%**, nearly tripling the improvement. This indicates that blindly searching for top- k continuations (as in Beam Search) is less effective than ADADEC's approach of triggering lookahead only at high-uncertainty points, which allows for more targeted and semantically meaningful corrections.

Comparison with AdapT. When compared to AdapT, ADADEC shows clear advantages in both stability and magnitude of improvement. While AdapT offers marginal improvements over Greedy decoding (e.g., +1.42% on HumanEval+ and +0.59% on MBPP+), it exhibits instability, leading to performance drops on DevEval for several models (average -0.47%). In contrast, ADADEC consistently achieves larger gains, surpassing AdapT by wide margins across nearly all models and datasets. For example, on MBPP+, ADADEC exceeds AdapT by approximately **+7%** on average. These results confirm that utilizing Shannon entropy to guide lookahead is a more reliable strategy than the heuristic-based temperature adjustment used in AdapT.

Summary: Overall, the results indicate that ADADEC is highly effective in improving the functional correctness of generated code. It consistently outperforms Greedy decoding across diverse models and benchmarks. More importantly, it surpasses Beam Search, particularly on complex tasks (MBPP+ and DevEval), proving that uncertainty-guided intervention is superior to blind search. Compared with AdapT, ADADEC demonstrates significantly more robust and scalable gains.

4.3 RQ2: Efficiency in Computational Cost

We evaluate the efficiency of ADADEC in terms of computational cost on the HumanEval+, MBPP+, and DevEval datasets. We analyze two key metrics: the *pause rate* and *prediction latency*.

Pause Rate. Table 3 presents the pause rates across different models and datasets. The results show that ADADEC triggers the reranking mechanism sparingly, with pause rates ranging from 0.85% to 12.17%, averaging approximately **6.75%** across all configurations. This low activation frequency indicates that the entropy-based threshold successfully identifies a small subset of critical, high-uncertainty steps. Unlike global search strategies, ADADEC concentrates computational effort only where necessary. Notably, even for the largest models, the pause rate generally remains below 10%, demonstrating that our selective triggering mechanism scales efficiently without incurring the cost of constant lookahead.

Latency. Table 4 reports the average generation time per problem for Greedy, Beam Search ($B = 3$), AdapT, and ADADEC.

Table 3. Pause ratio of the models on HumanEval+, MBPP+, and DevEval.

	DS-1.3B	DS-6.7B	ST-3B	QW2.5-1.5B	QW2.5-7B	QW3-0.6B	QW3-1.7B	QW3-4B	QW3-8B
HumanEval+	2.81%	2.33%	4.80%	1.48%	0.85%	6.11%	11.17%	7.82%	5.98%
MBPP+	8.49%	7.46%	6.52%	6.24%	4.43%	8.50%	12.17%	7.11%	8.33%
DevEval	7.75%	6.29%	6.59%	6.95%	9.70%	6.16%	11.47%	9.69%	5.04%
AVG. Δ	6.35%	5.36%	5.97%	4.89%	4.99%	6.92%	11.60%	8.21%	6.45%

Comparison with Greedy. Overall, ADADEC introduces moderate additional latency compared to Greedy decoding (e.g., an average increase of +0.75s on HumanEval+ and +0.32s on MBPP+). This overhead is expected due to the lookahead computations. However, considering the substantial accuracy gains reported in RQ1, this trade-off is highly favorable.

Notably, for certain models (e.g., QW3-0.6B and QW3-1.7B on HumanEval+), ADADEC actually exhibits lower latency than Greedy decoding. This counter-intuitive result occurs because ADADEC prevents the model from drifting into erroneous, low-quality trajectories—such as repetitive loops or verbose, nonsensical outputs—that often plague Greedy decoding at high-uncertainty points. By correcting the generation path early via lookahead, ADADEC produces more concise and correct solutions, thereby reducing the total number of generated tokens and effectively offsetting the computational cost of reranking.

Comparison with Beam Search. A critical finding is that ADADEC is significantly more efficient than Beam Search, despite outperforming it in accuracy. As shown in Table 4, Beam Search introduces substantial latency overhead due to maintaining multiple active beams at every step. For instance, on the repository-level DevEval benchmark, Beam Search increases the average generation time by +24.62s compared to Greedy. In contrast, ADADEC increases it by only +7.47s. This efficiency stems from our **selective pausing strategy**: by triggering the lookahead mechanism only at high-uncertainty steps rather than globally, ADADEC avoids wasted computation on trivial tokens.

Comparison with AdapT. AdapT exhibits mixed runtime behavior. In simpler scenarios (e.g., MBPP+), it adds negligible overhead (+0.10s) or even reduces time slightly for specific models by promoting shorter sequences via sampling. However, ADADEC provides a more consistent balance: while slightly slower than AdapT, it delivers far more robust functional correctness (as shown in RQ1), justifying the marginal increase in runtime.

Summary: ADADEC maintains high efficiency by selectively triggering lookahead at a low rate (average $\approx 6.75\%$), effectively filtering out low-utility computations. Crucially, compared to Beam Search, ADADEC incurs significantly lower additional overhead while achieving superior output quality across all benchmarks. This confirms that uncertainty-guided lookahead represents a far more efficient allocation of inference resources than blind search.

4.4 RQ3: Ablation Study

We perform an ablation study to evaluate the key components of ADADEC. In particular, we investigate whether the learned entropy threshold τ^{LM} serves as an effective signal, both in terms of predictive ability (as a classifier) and in practical decoding performance.

Predictive Effectiveness of Learned Entropy Thresholds. Table 5 summarizes the performance of the logistic regression classifier trained with learned entropy thresholds (τ^{LM}). Overall, the classifier demonstrates robust predictive capability across all models, achieving an average F1 score of 0.9351 and an AUC of 0.9116. The highest performance is observed on DS-6.7B, with an accuracy of 91.05% and an F1 score of 0.9512. Even for smaller models, the classifier remains

Table 4. Comparison of the average generation time per problem (in seconds) by Greedy, Beam Search, AdapT, ADADEC on HumanEval+, MBPP+, DevEval.

Model	HumanEval+				MBPP+				DevEval			
	Greedy	Beam	AdapT	ADADEC	Greedy	Beam	AdapT	ADADEC	Greedy	Beam	AdapT	ADADEC
DS-1.3B	2.94	7.01	2.72	4.56	1.46	1.91	1.47	1.68	4.03	14.59	4.24	7.66
DS-6.7B	9.55	25.60	13.59	11.87	3.21	7.51	3.13	4.81	20.56	53.52	23.72	32.91
ST-3B	1.71	3.45	2.72	2.06	0.60	1.01	0.84	0.77	6.92	34.40	8.33	14.90
QW2.5-1.5B	5.29	9.39	4.18	5.82	1.78	2.24	1.55	2.24	7.91	19.02	7.88	10.12
QW2.5-7B	12.24	38.22	13.40	14.38	3.94	10.12	3.90	6.02	12.12	26.51	13.71	22.19
QW3-0.6B	6.34	8.94	5.18	5.03	5.94	9.44	6.14	3.89	19.49	23.79	17.86	21.84
QW3-1.7B	8.23	9.98	6.03	5.29	2.13	3.17	2.54	2.28	22.21	46.98	20.56	29.53
QW3-4B	5.41	9.13	7.29	8.10	3.49	5.02	3.76	3.37	29.57	80.24	31.94	42.78
QW3-8B	5.62	11.76	5.63	6.99	3.88	5.19	4.03	4.21	61.14	106.48	63.53	69.23
AVG. Δ	–	+7.35	+0.38	+0.75	–	+2.13	+0.10	+0.32	–	+24.62	+0.87	+7.47

reliable; for instance, ST-3B achieves an F1 score of 0.9475. These high metrics confirm that Shannon entropy, when calibrated via our learning process, serves as a high-fidelity signal for distinguishing between correct and incorrect token predictions.

Table 5. Evaluation of logistic regression models trained to predict whether the ground truth token is the top-1 prediction. The learned τ^{LM} denotes the entropy threshold selected via logistic regression.

Model	Accuracy	Precision	Recall	F1 Score	AUC	Learned τ^{LM}
DS-1.3B	0.8994	0.9251	0.9640	0.9442	0.9268	0.9850
DS-6.7B	0.9105	0.9276	0.9761	0.9512	0.9305	1.1338
ST-3B	0.9038	0.9188	0.9780	0.9475	0.9222	1.1005
QW2.5-1.5B	0.8824	0.9055	0.9638	0.9337	0.9085	1.2232
QW2.5-7B	0.8889	0.9106	0.9679	0.9384	0.9119	1.1739
QW3-0.6B	0.8512	0.8774	0.9498	0.9122	0.8914	1.3213
QW3-1.7B	0.8664	0.9003	0.9457	0.9225	0.8964	0.6153
QW3-4B	0.8807	0.9166	0.9471	0.9316	0.9073	0.7134
QW3-8B	0.8843	0.9158	0.9533	0.9342	0.9092	0.7060
AVG. Δ	0.8853	0.9109	0.9606	0.9351	0.9116	0.9969

Practical Effectiveness of Learned Entropy Thresholds. To validate the necessity of learning the threshold rather than using a heuristic constant, we compare the decoding performance of models using the learned threshold versus a fixed threshold. The fixed threshold is set to 1.2. Table 6 reports the Pass@1 results on HumanEval+.

The experiments demonstrate that using the learned thresholds yields consistent improvements over the fixed threshold baseline across all evaluated models. The average improvement is **+1.76%**, with gains ranging from +0.61% to +4.88%. Specifically, while most models (e.g., DS-1.3B, QW2.5-7B, QW3-8B) show stable moderate gains between 1.2% and 1.8%, distinctively larger boosts are observed for QW3-1.7B (+3.05%) and QW3-4B (+4.88%). This indicates that a fixed threshold of 1.2 is often suboptimal. By calibrating the threshold to each model’s specific uncertainty distribution,

ADADec ensures robust performance gains, effectively unlocking the model’s potential regardless of its specific architecture or scale.

Table 6. Comparison of Learned vs. Fixed Entropy Thresholds on HumanEval+ (Pass@1). Fixed threshold is set to 1.2.

Strategy	DS-1.3B	DS-6.7B	ST-3B	QW2.5-1.5B	QW2.5-7B	QW3-0.6B	QW3-1.7B	QW3-4B	QW3-8B	AVG
Fixed	63.41%	73.17%	50.00%	65.85%	85.98%	21.34%	41.46%	51.22%	61.59%	57.11%
Learned	65.24%	73.78%	50.61%	67.07%	87.20%	22.56%	44.51%	56.10%	62.80%	58.87%
Δ	+1.83%	+0.61%	+0.61%	+1.22%	+1.22%	+1.22%	+3.05%	+4.88%	+1.21%	+1.76%

Summary: The learned entropy threshold τ^{LM} is both theoretically sound and practically effective. It reliably predicts token risks and consistently outperforms a fixed-threshold baseline across all models, delivering an average Pass@1 improvement of +1.76% and maximizing the effectiveness of the uncertainty-guided intervention.

4.5 RQ4: Parameter Sensitivity

To investigate the impact of key hyperparameters on decoding performance, we conduct sensitivity analyses on the lookahead length L and the learned entropy threshold τ^{LM} . All experiments in this section use DS-1.3B as the backbone model evaluated on HumanEval+.

Sensitivity to Lookahead Length (L). We vary L from 2 to 9 and report Pass@1 results. Figure 6 illustrates the trend.

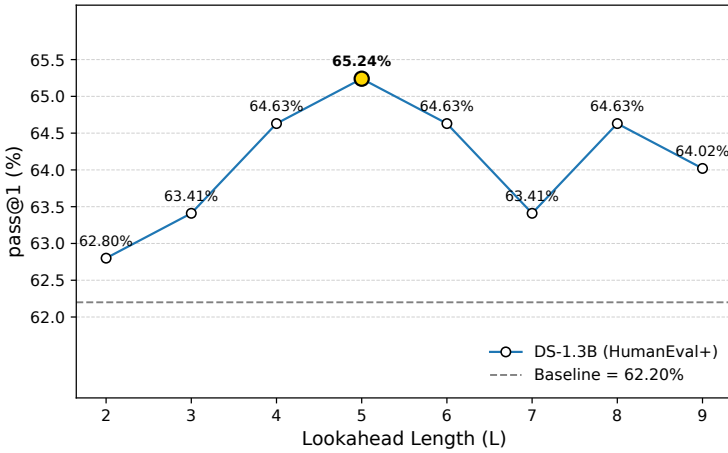


Fig. 6. Pass@1 Performance of the DS-1.3B Model on the HumanEval+ Dataset Under Different Lookahead Length (L) Configurations.

The results show that Pass@1 increases steadily with larger L up to $L = 5$, at which point it reaches the peak performance of 65.24%. Beyond $L = 5$, the performance does not continue to rise; instead, it exhibits slight fluctuations, hovering between 63.41% and 64.63%. This behavior reflects the trade-off between local guidance and global noise: while shorter lookaheads may fail to capture sufficient future context, excessively long lookaheads ($L > 5$) may introduce noise from uncertainty accumulation, diluting the relevance of immediate local signals.

Crucially, across all values of L , ADADEC consistently outperforms the Greedy baseline (62.20%). Even at the minimal setting of $L = 2$, ADADEC achieves 62.80%, reducing to a safe margin of +0.60%. This demonstrates that our approach is robust to hyperparameter choices and provides stable improvements.

Sensitivity to Threshold Variations (τ^{LM}). As illustrated in Table 7, we evaluate the Pass@1 accuracy by shifting the optimal learned threshold ($\tau^{LM} \approx 0.9850$) across various intervals. ADADEC demonstrates high robustness to minor fluctuations. Within the $[\tau^{LM} - 0.1, \tau^{LM} + 0.1]$ range, Pass@1 remains highly stable and consistently outperforms the greedy baseline.

Table 7. Sensitivity of ADADEC performance (Pass@1) to entropy threshold (τ) variations on HumanEval+. Evaluated with DS-1.3B. **Learned** denotes the optimal threshold derived from our approach.

Setting	Threshold τ	Pass@1 (%)	Pause Rate (%)
Baseline (Greedy)	–	62.20	–
$\tau^{LM} - 0.50$	0.4850	63.41	7.82
$\tau^{LM} - 0.10$	0.8850	65.24	3.46
$\tau^{LM} - 0.05$	0.9350	64.63	3.10
τ^{LM} (Learned)	0.9850	65.24	2.81
$\tau^{LM} + 0.05$	1.0350	65.24	2.52
$\tau^{LM} + 0.10$	1.0850	64.63	2.16
$\tau^{LM} + 0.50$	1.4850	62.80	0.83

Visible performance degradation occurs only under drastic shifts (± 0.5): excessively low thresholds introduce noise via over-intervention, while excessively high thresholds become too conservative. This confirms that τ^{LM} is not a fragile hyperparameter requiring extreme precision.

Summary: Both key hyperparameters exhibit high robustness. The lookahead length L shows a non-monotonic effect, with moderate values (e.g., $L = 5$) maximizing the benefits of lookahead guidance. The learned threshold τ^{LM} is highly stable against minor fluctuations within ± 0.1 . Notably, ADADEC consistently surpasses baseline performance across varying hyperparameter settings.

5 Discussion

5.1 Generalizability and Transferability of the Learned Threshold (τ^{LM})

We conduct a quantitative analysis using DS-1.3B to evaluate the robustness of the learned entropy threshold (τ^{LM}) across domain and prompt shifts, and provide guidance on its practical application.

Robustness to Domain and Prompt Shifts. We evaluate the transferability of τ^{LM} to library-heavy domains and varying prompt templates. We compute the optimal thresholds for two specific data science scenarios: a data science subset filtered from BigCodeBench (666 problems) and the DS-1000 benchmark [17] (1000 problems, featuring drastically different prompt styles and task requirements). The optimal thresholds for these two datasets are 0.9740 ($\Delta = 0.0110$) and 0.9178 ($\Delta = 0.0672$), respectively. Since our parameter sensitivity analysis (Section 4.5) demonstrates that deviations within ± 0.1 do not significantly impact performance, a globally learned τ^{LM} remains highly effective without the need for task-specific tuning.

Guidance on Relearning τ^{LM} . These findings indicate that τ^{LM} reflects the intrinsic uncertainty calibration of the base LLM rather than task-specific artifacts. Therefore, relearning τ^{LM} is generally unnecessary for different standard coding domains or prompt styles. Retraining is only

recommended when the base model’s confidence distribution changes fundamentally, such as after Supervised Fine-Tuning (SFT) or alignment techniques.

5.2 Limitations

While ADADEC demonstrates improvements in code generation accuracy and maintains acceptable computational cost, it has several limitations that warrant further investigation:

Reliance Solely on Shannon Entropy. Our uncertainty trigger is based solely on the Shannon entropy of the token distribution, which provides a coarse measure of model uncertainty. More nuanced contextual features—such as entropy of subpopulations of tokens, token-level gradient norms, or higher-order statistics—might better capture subtle divergences. Future work could integrate richer uncertainty signals or leverage learned uncertainty estimators.

Threshold Learning Simplicity. We employ a logistic regression classifier to learn entropy thresholds, which offers interpretability but may be insufficiently expressive for capturing complex interactions among uncertainty features. More powerful models (e.g., small neural networks) could potentially improve threshold accuracy, at the cost of increased complexity.

Fixed Lookahead Depth. Currently, ADADEC utilizes a fixed lookahead length (e.g., $L = 5$) across all benchmarks. While this serves as a reasonable global setting, repository-level tasks (e.g., DevEval) often require longer-range reasoning, and a short lookahead window may under-express these dependencies. Future research could explore adaptive, context-aware lookahead depths that dynamically adjust to accommodate both short-term syntactic constraints and long-term semantic reasoning demands.

Addressing these limitations will be an actionable direction for future work, towards making uncertainty-guided adaptive decoding more robust, flexible, and broadly applicable.

5.3 Case Study

To illustrate ADADEC’s practical advantage, Figure 7 presents the `check_dict_case` task from HumanEval+. While standard greedy decoding unsafely assumes all dictionary keys are strings (causing runtime errors for integer keys), ADADEC detects high entropy at this critical decision point. Consequently, it triggers lookahead reranking and selects a robust continuation that explicitly verifies key types (e.g., `isinstance(key, str)`) before case checking. This demonstrates how ADADEC leverages entropy signals to proactively prevent subtle bugs and enhance generation reliability.

6 Threats to Validity

We analyze threats to validity across four dimensions—internal, external, construct, and conclusion—and note our mitigation strategies or key limitations.

Internal Validity. Drift point identification may be imperfect: our semi-automated AST comparison could miss subtle semantic shifts (e.g., identifier drift), and human verification introduces potential annotator bias. We mitigated these risks through clear guidelines and cross-checking. Entropy-based analysis may also depend on tokenizer or model-specific distributions; to reduce this risk, we validated results across multiple models and confirmed entropy–rank correlations in RQ2.

External Validity. We focus on open-source models (DeepSeek, StableCode, Qwen) and three benchmarks (HumanEval+, MBPP+, DevEval). Results may not fully generalize to proprietary models or to broader real-world tasks. Future work should extend evaluation to more diverse datasets and model families.

Construct Validity. We evaluate functional correctness (Pass@1) and efficiency (latency), which directly measure the primary objectives of correctness and performance. Nevertheless, these metrics

	<pre>def check_dict_case(dict): """ Given a dictionary, return True if all keys are strings in lower case or all keys are strings in upper case, else return False. The function should return False is the given dictionary is empty. Examples: check_dict_case({"a":"apple", "b":"banana"}) should return True. check_dict_case({"a":"apple", "A":"banana", "B":"banana"}) should return False. check_dict_case({"a":"apple", 8:"banana", "a":"apple"}) should return False. check_dict_case({"Name":"John", "Age":"36", "City":"Houston"}) should return False. check_dict_case({"STATE":"NC", "ZIP":"12345" }) should return True. """ if not dict: return False keys = dict.keys()</pre>	Prompt
Next Step	<pre>Entropy:2.1953 > Threshold Top3 Tokens: ['if', 'all', 'lower']</pre>	Current Seq
	<pre>all_lower = all(k.islower() for k in keys) all_upper = all(k.isupper() for k in keys) return all_lower or all_upper</pre>	Greedy 
	<pre>if all(isinstance(key, str) for key in keys): if all(key.islower() for key in keys): return True elif all(key.isupper() for key in keys): return True return False</pre>	AdaDec 

Fig. 7. A case study from HumanEval+

do not capture other important qualities such as readability or adherence to coding conventions. Similarly, our use of Shannon entropy as an uncertainty signal assumes it faithfully reflects model uncertainty, though it may only provide a coarse approximation. Alternative measures could offer complementary insights.

Conclusion Validity. Conclusion validity threats arise from the statistical robustness of our empirical results. Although we report consistent trends across multiple models and datasets, the sample size of benchmarks such as HumanEval+ remains relatively small, which may limit the statistical power of our findings. Additionally, hyperparameter choices (e.g., lookahead length L , candidate pool size B) may affect performance outcomes, and while we explored reasonable ranges, further sensitivity analysis would strengthen confidence in our conclusions.

7 Related Work

7.1 LLM-based Code Generation

Large Language Models (LLMs) have shown strong potential in software engineering tasks such as code generation [4, 14, 20, 22, 37], unit test generation [25, 28, 33, 40, 42], and comment generation [9, 16, 26]. Typically, LLMs generate code from textual requirements via prompt-based interaction. By learning expressive representations from large-scale code and documentation corpora, they can produce syntactically correct and context-aware code snippets [35, 38, 41]. Open-foundation models further advance tasks like code completion, bug fixing [7, 15, 39], and test-case generation.

To evaluate their effectiveness, benchmarks like HumanEval [3], MBPP [1], and DS-1000 [17] focus on function-level generation, while ClassEval [5] extends evaluation to class-level tasks [2].

More recently, CoderEval [43] and DevEval [19] target repository-level scenarios, reflecting growing research interest in applying LLMs to real-world software development.

7.2 Decoding strategies for LLMs

Large Language Models (LLMs) generate text in an autoregressive manner, producing one token at a time. At each generation step, the model samples a token from its predicted probability distribution. The most straightforward sampling strategy is greedy decoding, which selects the token with the highest probability as the next word. A more refined variant is beam search [6], which maintains multiple candidate sequences at each decoding step and preserves the top-k sequences based on cumulative log-probabilities.

Beyond these, commonly used sampling strategies include temperature sampling [10], top-p (nucleus) sampling [10], and top-k sampling [10], each designed to balance diversity and coherence in generated outputs.

To further improve sampling for code generation tasks, Li et al. introduced AdapT Sampling [45] (Adaptive Temperature Sampling). This method dynamically adjusts the temperature parameter T based on token context. For instance, when the current token initiates a code block, a higher T is used to encourage diversity and creativity. In contrast, for other tokens, a lower T is applied to suppress random noise and preserve syntactic and semantic correctness.

Another relevant approach is UnCert-CoT [44], which enhances code generation through uncertainty-aware Chain-of-Thought (CoT) reasoning, activating multi-path reasoning when token uncertainty surpasses a fixed threshold. While effective, it relies on fixed thresholds and costly CoT reasoning. In contrast, ADADEC learns model-specific uncertainty thresholds for stronger generalization, while its lightweight rerank mechanism cuts computational costs significantly without compromising benchmark performance.

8 Conclusion

This paper presents ADADEC, an uncertainty-guided adaptive decoding framework for large language model-based code generation. Motivated by an empirical study revealing that ranking errors at high-uncertainty decoding steps often lead to logic drift, ADADEC introduces a novel pause-then-rerank mechanism that dynamically identifies and corrects uncertain token predictions using Shannon entropy. By learning model-specific entropy thresholds and employing a lookahead-based reranking strategy, ADADEC achieves substantial improvements in accuracy while maintaining computational cost and efficiency within acceptable bounds. Experiments on HumanEval+, MBPP+, and DevEval benchmarks demonstrate that ADADEC consistently outperforms all compared methods. These findings underscore the importance of token-level uncertainty modeling and open up new directions for adaptive decoding in code generation and beyond.

9 Data Availability

To facilitate the replication study, we have released our data and code at : <https://github.com/SYSUSELab/AdaDec>.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (Grant No. 92582117, Grant No. 62402113), the General Program of the Natural Science Foundation of Guangdong Province, China (Grant No. 2025A1515011631), and GMCC-SYSU Joint Lab for Smart Applications.

References

- [1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. arXiv:2108.07732 [cs.PL] <https://arxiv.org/abs/2108.07732>
- [2] Jialun Cao, Yuk-Kit Chan, Zixuan Ling, Wenxuan Wang, Shuqing Li, Mingwei Liu, Ruixi Qiao, Yuting Han, Chaozheng Wang, Boxi Yu, Pinjia He, Shuai Wang, Zibin Zheng, Michael R. Lyu, and Shing-Chi Cheung. 2026. Rigor, Reliability, and Reproducibility Matter: A Decade-Scale Survey of 572 Code Benchmarks. arXiv:2501.10711 [cs.SE] <https://arxiv.org/abs/2501.10711>
- [3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebggen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] <https://arxiv.org/abs/2107.03374>
- [4] Yifeng Di and Tianyi Zhang. 2025. Enhancing Code Generation via Bidirectional Comment-Level Mutual Grounding. arXiv:2505.07768 [cs.SE] <https://arxiv.org/abs/2505.07768>
- [5] Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2023. ClassEval: A Manually-Crafted Benchmark for Evaluating LLMs on Class-level Code Generation. arXiv:2308.01861 [cs.CL] <https://arxiv.org/abs/2308.01861>
- [6] Markus Freitag and Yaser Al-Onaizan. 2017. Beam Search Strategies for Neural Machine Translation. In *Proceedings of the First Workshop on Neural Machine Translation*, Thang Luong, Alexandra Birch, Graham Neubig, and Andrew Finch (Eds.). Association for Computational Linguistics, Vancouver, 56–60. doi:10.18653/v1/W17-3207
- [7] Michael Fu, Chakkrit Tantithamthavorn, Trung Le, Van Nguyen, and Dinh Phung. 2022. VulRepair: a T5-based automated software vulnerability repair. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Singapore, Singapore) (ESEC/FSE 2022)*. Association for Computing Machinery, New York, NY, USA, 935–947. doi:10.1145/3540250.3549098
- [8] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y.K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. <https://arxiv.org/abs/2401.14196>
- [9] Md. Asif Haider, Ayesha Binte Mostofa, Sk. Sabit Bin Mosaddek, Anindya Iqbal, and Toufique Ahmed. 2024. Prompting and Fine-tuning Large Language Models for Automated Code Review Comment Generation. arXiv:2411.10129 [cs.SE] <https://arxiv.org/abs/2411.10129>
- [10] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2020. The Curious Case of Neural Text Degeneration. arXiv:1904.09751 [cs.CL] <https://arxiv.org/abs/1904.09751>
- [11] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. 2024. Qwen2.5-Coder Technical Report. arXiv:2409.12186 [cs.CL] <https://arxiv.org/abs/2409.12186>
- [12] Nam Huynh and Beiyu Lin. 2025. Large Language Models for Code Generation: A Comprehensive Survey of Challenges, Techniques, Evaluation, and Applications. arXiv:2503.01245 [cs.SE] <https://arxiv.org/abs/2503.01245>
- [13] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A Survey on Large Language Models for Code Generation. arXiv:2406.00515 [cs.CL] <https://arxiv.org/abs/2406.00515>
- [14] Xue Jiang, Yihong Dong, Yongding Tao, Huanyu Liu, Zhi Jin, Wenpin Jiao, and Ge Li. 2025. RCODE: Integrating Backtracking Mechanism and Program Analysis in Large Language Models for Code Generation. arXiv:2411.07112 [cs.SE] <https://arxiv.org/abs/2411.07112>
- [15] Matthew Jin, Syed Shahriar, Michele Tufano, Xin Shi, Shuai Lu, Neel Sundaresan, and Alexey Svyatkovskiy. 2023. InferFix: End-to-End Program Repair with LLMs. arXiv:2303.07263 [cs.SE] <https://arxiv.org/abs/2303.07263>
- [16] Jonathan Katzy, Yongcheng Huang, Gopal-Raj Panchu, Maksym Ziemlewski, Paris Loizides, Sander Vermeulen, Arie van Deursen, and Maliheh Izadi. 2025. A Qualitative Investigation into LLM-Generated Multilingual Code Comments and Automatic Evaluation Metrics. arXiv:2505.15469 [cs.SE] <https://arxiv.org/abs/2505.15469>
- [17] Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Scott Wen tau Yih, Daniel Fried, Sida Wang, and Tao Yu. 2022. DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation. arXiv:2211.11501 [cs.SE] <https://arxiv.org/abs/2211.11501>

- [18] Guillaume Lemaître, Fernando Nogueira, and Christos K. Aridas. 2016. Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning. arXiv:1609.06570 [cs.LG] <https://arxiv.org/abs/1609.06570>
- [19] Jia Li, Ge Li, Yunfei Zhao, Yongmin Li, Huanyu Liu, Hao Zhu, Lecheng Wang, Kaibo Liu, Zheng Fang, Lanshen Wang, Jiazheng Ding, Xuanming Zhang, Yuqi Zhu, Yihong Dong, Zhi Jin, Binhua Li, Fei Huang, and Yongbin Li. 2024. DevEval: A Manually-Annotated Code Generation Benchmark Aligned with Real-World Code Repositories. arXiv:2405.19856 [cs.CL] <https://arxiv.org/abs/2405.19856>
- [20] Jia Li, Yongmin Li, Ge Li, Zhi Jin, Yiyang Hao, and Xing Hu. 2023. SkCoder: A Sketch-based Approach for Automatic Code Generation. arXiv:2302.06144 [cs.SE] <https://arxiv.org/abs/2302.06144>
- [21] Zike Li, Mingwei Liu, Anji Li, Kaifeng He, Yanlin Wang, Xin Peng, and Zibin Zheng. 2025. A Preliminary Study on the Robustness of Code Generation by Large Language Models. arXiv:2503.20197 [cs.SE] <https://arxiv.org/abs/2503.20197>
- [22] Feng Lin, Dong Jae Kim, Tse-Husn, and Chen. 2024. SOEN-101: Code Generation by Emulating Software Process Models Using Large Language Model Agents. arXiv:2403.15852 [cs.SE] <https://arxiv.org/abs/2403.15852>
- [23] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. arXiv:2305.01210 [cs.SE] <https://arxiv.org/abs/2305.01210>
- [24] Sicong Liu, Yanxian Huang, Mingwei Liu, Jiachi Chen, Ensheng Shi, Yuchi Ma, Hongyu Zhang, Yin Zhang, and Yanlin Wang. 2026. ShortCoder: Knowledge-Augmented Syntax Optimization for Token-Efficient Code Generation. arXiv:2601.09703 [cs.SE] <https://arxiv.org/abs/2601.09703>
- [25] Andrea Lops, Fedelucio Narducci, Azzurra Ragone, Michelantonio Trizio, and Claudio Bartolini. 2025. A System for Automated Unit Test Generation using Large Language Models and Assessment of Generated Test Suites. In *2025 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE, 29–36. doi:10.1109/icstw64639.2025.10962454
- [26] Hanzhen Lu and Zhongxin Liu. 2024. Improving Retrieval-Augmented Code Comment Generation by Retrieving for Generation. arXiv:2408.03623 [cs.SE] <https://arxiv.org/abs/2408.03623>
- [27] Ximing Lu, Sean Welleck, Peter West, Liwei Jiang, Jungo Kasai, Daniel Khashabi, Ronan Le Bras, Lianhui Qin, Youngjae Yu, Rowan Zellers, Noah A. Smith, and Yejin Choi. 2021. NeuroLogic A'esque Decoding: Constrained Text Generation with Lookahead Heuristics. arXiv:2112.08726 [cs.CL] <https://arxiv.org/abs/2112.08726>
- [28] Zifan Nan, Zhaoqiang Guo, Kui Liu, and Xin Xia. 2025. Test Intention Guided LLM-based Unit Test Generation . In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, Los Alamitos, CA, USA, 779–779. doi:10.1109/ICSE55347.2025.00243
- [29] Nikhil Pinnaparaju, Reshith Adithyan, Duy Phung, Jonathan Tow, James Baicoianu, and Nathan Cooper. 2023. Stable Code 3B. <https://huggingface.co/stabilityai/stable-code-3b>.
- [30] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. *Language Models are Unsupervised Multitask Learners*. Technical Report. OpenAI. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf Accessed: 2024-11-15.
- [31] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. arXiv:1908.10084 [cs.CL] <https://arxiv.org/abs/1908.10084>
- [32] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2024. Code Llama: Open Foundation Models for Code. arXiv:2308.12950 [cs.CL] <https://arxiv.org/abs/2308.12950>
- [33] Gabriel Ryan, Siddhartha Jain, Mingyue Shang, Shiqi Wang, Xiaofei Ma, Murali Krishna Ramanathan, and Baishakhi Ray. 2024. Code-Aware Prompting: A Study of Coverage-Guided Test Generation in Regression Setting using LLM. *Proc. ACM Softw. Eng.* 1, FSE, Article 43 (July 2024), 21 pages. doi:10.1145/3643769
- [34] C. E. Shannon. 1948. A mathematical theory of communication. *The Bell System Technical Journal* 27, 3 (1948), 379–423. doi:10.1002/j.1538-7305.1948.tb01338.x
- [35] Zhihong Sun, Chen Lyu, Bolun Li, Yao Wan, Hongyu Zhang, Ge Li, and Zhi Jin. 2024. Enhancing Code Generation Performance of Smaller Models by Distilling the Reasoning Ability of LLMs. arXiv:2403.13271 [cs.SE] <https://arxiv.org/abs/2403.13271>
- [36] Qwen Team. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] <https://arxiv.org/abs/2505.09388>
- [37] Zhao Tian, Junjie Chen, and Xiangyu Zhang. 2024. Fixing Large Language Models' Specification Misunderstanding for Better Code Generation. arXiv:2309.16120 [cs.SE] <https://arxiv.org/abs/2309.16120>
- [38] Shubham Ugare, Tarun Suresh, Hangoo Kang, Sasa Misailovic, and Gagandeep Singh. 2024. SynCode: LLM Generation with Grammar Augmentation. arXiv:2403.01632 [cs.LG] <https://arxiv.org/abs/2403.01632>
- [39] Chunqiu Steven Xia and Lingming Zhang. 2022. Less training, more repairing please: revisiting automated program repair via zero-shot learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and*

- Symposium on the Foundations of Software Engineering* (Singapore, Singapore) (*ESEC/FSE 2022*). Association for Computing Machinery, New York, NY, USA, 959–971. doi:10.1145/3540250.3549101
- [40] Lin Yang, Chen Yang, Shutao Gao, Weijing Wang, Bo Wang, Qihao Zhu, Xiao Chu, Jianyi Zhou, Guangtai Liang, Qianxiang Wang, and Junjie Chen. 2024. On the Evaluation of Large Language Models in Unit Test Generation. arXiv:2406.18181 [cs.SE] <https://arxiv.org/abs/2406.18181>
- [41] Zhiqiang Yuan, Junwei Liu, Qiancheng Zi, Mingwei Liu, Xin Peng, and Yiling Lou. 2023. Evaluating Instruction-Tuned Large Language Models on Code Comprehension and Generation. arXiv:2308.01240 [cs.CL] <https://arxiv.org/abs/2308.01240>
- [42] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. 2024. Evaluating and Improving ChatGPT for Unit Test Generation. *Proc. ACM Softw. Eng.* 1, FSE, Article 76 (July 2024), 24 pages. doi:10.1145/3660783
- [43] Yakun Zhang, Wenjie Zhang, Dezhi Ran, Qihao Zhu, Chengfeng Dou, Dan Hao, Tao Xie, and Lu Zhang. 2024. Learning-based Widget Matching for Migrating GUI Test Cases. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) (*ICSE '24*). Association for Computing Machinery, New York, NY, USA, Article 69, 13 pages. doi:10.1145/3597503.3623322
- [44] Yuqi Zhu, Ge Li, Xue Jiang, Jia Li, Hong Mei, Zhi Jin, and Yihong Dong. 2025. Uncertainty-Guided Chain-of-Thought for Code Generation with LLMs. arXiv:2503.15341 [cs.SE] <https://arxiv.org/abs/2503.15341>
- [45] Yuqi Zhu, Jia Li, Ge Li, Yunfei Zhao, Jia Li, Zhi Jin, and Hong Mei. 2023. Hot or Cold? Adaptive Temperature Sampling for Code Generation with Large Language Models. arXiv:2309.02772 [cs.SE] <https://arxiv.org/abs/2309.02772>
- [46] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widayarsi, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, Simon Brunner, Chen Gong, Thong Hoang, Arnel Randy Zebaze, Xiaoheng Hong, Wen-Ding Li, Jean Kaddour, Ming Xu, Zhihan Zhang, Prateek Yadav, Naman Jain, Alex Gu, Zhoujun Cheng, Jiawei Liu, Qian Liu, Zijian Wang, Binyuan Hui, Niklas Muennighoff, David Lo, Daniel Fried, Xiaoning Du, Harm de Vries, and Leandro Von Werra. 2025. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. arXiv:2406.15877 [cs.SE] <https://arxiv.org/abs/2406.15877>

Received 2025-08-30; accepted 2026-03-24